



Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра автоматизации систем вычислительных комплексов

Гоцкало Максим Олегович

**Детерминированные алгоритмы построения
многопроцессорного расписания
с ограничением на объем памяти**

Курсовая работа

Научный руководитель
к.ф.-м.н., с.н.с.
Балашов Василий Викторович

Москва, 2026

Аннотация

Здесь будет аннотация

Содержание

Введение	5
1 Цель работы	6
2 Формальная постановка задачи	7
2.1 Модель прикладной программы	7
2.2 Модель вычислительной системы	7
2.3 Модель использования памяти	8
2.4 Модель расписания	8
2.5 Целевая функция	9
3 Жадный алгоритм построения многопроцессорного расписания с ограничением на память	10
3.1 Формальное описание алгоритма	10
3.2 Эвристики выбора работы	12
3.2.1 Эвристика EFT	12
3.2.2 Эвристика STS	13
4 Сведение к задаче целочисленного линейного программирования	14
5 Экспериментальное исследование	19
5.1 Предмет и цели исследования	19
5.2 Классы графов работ	19
5.2.1 Слоистые графы	19
5.2.2 Случайные графы	19
5.2.3 Треугольные графы	20
5.3 Наборы входных данных	20
5.3.1 Малые графы	21
5.3.2 Большие графы	21
5.3.3 Сетка параметров (P, M)	21
5.4 Результаты экспериментов	22
5.4.1 Малые графы	23

6	Описание программной реализации	27
6.1	Структура входного и выходного файлов	27
6.1.1	Входной файл (описание графа работ)	27
6.1.2	Выходной файл (расписание)	28
6.2	Реализация жадного алгоритма	29
6.3	Реализация подхода на базе ЦЛП	32
6.4	Визуализатор временной диаграммы расписания	33
	Заключение	36

Введение

Здесь будет введение

1 Цель работы

Целью работы является разработка детерминированных алгоритмов построения многопроцессорного расписания с ограничением на объем памяти в целевой системе и минимизацией длительности расписания.

Для достижения цели необходимо:

1. разработать жадный алгоритм и эвристики в его составе;
2. выполнить сведение задачи построения расписания к задаче целочисленного линейного программирования (ЦЛП);
3. провести экспериментальное исследование жадного алгоритма и подхода на основе ЦЛП по критериям качества решения и длительности его поиска.

2 Формальная постановка задачи

2.1 Модель прикладной программы

Модель прикладной программы задается ориентированным ациклическим графом потока данных

$$G = (V, E), \quad |V| = n, \quad |E| = m.$$

Каждой вершине $v \in V$ соответствует работа, а каждой дуге $(u, v) \in E$ — зависимость по данным между работами u и v . Если $(u, v) \in E$, то работа v может начать выполняться только после завершения работы u .

Для каждой работы $v \in V$ задана длительность исполнения $d_v > 0$. Кроме того, каждая работа может формировать один или несколько выходных буферов. Обозначим множество буферов, создаваемых работой v , через

$$B(v) = \{b_{v1}, b_{v2}, \dots, b_{vk}\}.$$

Для каждого буфера $b \in B(v)$ задаются его объем памяти $r_b > 0$ и множество потребителей $C(b) \subseteq V$. Если $u \in C(b)$, то работа u использует результат, содержащийся в буфере b .

Такая модель соответствует формату входных данных, где для каждой работы задаются ее длительность и набор зависимостей вида:

вес_буфера: список_потомков

2.2 Модель вычислительной системы

Вычислительная система состоит из P одинаковых процессоров

$$SP = \{sp_1, sp_2, \dots, sp_P\}.$$

Каждый процессор в любой момент времени может выполнять не более одной работы. Прерывания работ не допускаются, миграция уже запущенной работы между процессорами невозможна.

Отдельные задержки межпроцессорной передачи данных не учитываются. Поэтому момент готовности работы определяется завершением всех ее непосредственных предков. Ограничивающими факторами являются зависимости между работами, число доступных процессоров и общий объем памяти M , доступный для хранения активных буферов [? ?].

2.3 Модель использования памяти

Пусть работа v стартует в момент s_v и завершается в момент

$$f_v = s_v + d_v.$$

Буфер $b \in B(v)$ выделяется в момент старта работы v . Его освобождение происходит после завершения последнего из них:

$$fr_b = \max_{u \in C(b)} f_u.$$

Обозначим через $J(\tau)$ множество буферов, активных в момент времени τ . Тогда текущее использование памяти равно

$$\text{mem}(\tau) = \sum_{b \in J(\tau)} r_b.$$

Пиковое использование памяти на расписании HP определяется как

$$\text{peak}(HP) = \max_{\tau} (\text{mem}(\tau)).$$

Расписание считается допустимым по памяти, если выполнено условие

$$\text{peak}(HP) \leq M.$$

2.4 Модель расписания

Расписание HP задает для каждой работы $v \in V$:

1. номер процессора $p(v) \in \{1, \dots, P\}$;
2. момент старта s_v ;

3. момент завершения $f_v = s_v + d_v$.

Расписание является корректным, если выполняются следующие ограничения:

1. каждая работа назначена ровно одному процессору;
2. на каждом процессоре интервалы выполнения назначенных ему работ не пересекаются;
3. для каждой дуги $(u, v) \in E$ выполнено неравенство $f_u \leq s_v$;
4. для всего расписания выполнено ограничение по памяти $\text{peak}(HP) \leq M$.

2.5 Целевая функция

Минимизируемым критерием является длительность расписания, то есть момент завершения последней работы:

$$C_{\max}(HP) = \max_{v \in V} f_v.$$

Требуется найти такое допустимое расписание HP^* , что

$$C_{\max}(HP^*) = \min_{HP \in \mathcal{H}_{\text{adm}}} C_{\max}(HP),$$

где $\mathcal{H}_{\text{adm}}$ — множество всех корректных расписаний.

3 Жадный алгоритм построения многопроцессорного расписания с ограничением на память

Жадные алгоритмы (ЖА) широко применяются для построения расписаний в силу их низкой вычислительной сложности и детерминированности. В данной работе рассматривается адаптация списочного подхода к задаче, описанной в разделе 2, с дополнительным ограничением на суммарный объём используемой памяти. В отличие от классической постановки, где учитываются только зависимости по данным и доступные процессоры, здесь необходимо также контролировать, чтобы в любой момент времени суммарный объём занятых буферов не превышал заданный лимит M .

Предлагаемый жадный алгоритм основан на последовательном назначении работ, которые готовы к выполнению (т.е. все их предшественники уже поставлены в расписание). На каждом шаге для каждой готовой работы определяется наиболее раннее возможное время старта на каждом процессоре с учётом уже построенной части расписания, ограничений частичного порядка и лимита памяти. Затем с помощью заданной эвристики выбирается одна работа (и процессор, на котором достигнуто минимальное время старта), которая фиксируется в расписании. Процесс повторяется, пока все работы не будут назначены.

3.1 Формальное описание алгоритма

Пусть задан ориентированный ациклический граф работ $G = (V, E)$, где $V = \{1, \dots, n\}$ — множество работ, $E \subseteq V \times V$ — дуги, задающие отношение предшествования. Каждая работа i характеризуется:

- длительностью $d_i > 0$;
- количеством создаваемых буферов n_i ;
- объёмами буферов $r_{i1}, r_{i2}, \dots, r_{in_i}$.

Для каждой дуги $(i, k, j) \in E$ (работа k потребляет j -й буфер работы i) задано, какой именно буфер передаётся. Вычислительная система состоит из P одинаковых процессоров, каждый из которых может выполнять не более одной работы в каждый момент времени. Доступный объём памяти равен M .

Алгоритм работает следующим образом.

Шаг 1. Инициализация.

- Множество ещё не назначенных работ $\mathcal{U} = V$.
- Множество готовых работ $\mathcal{R} = \{i \in V \mid \text{у } i \text{ нет предшественников}\}$.
- Для каждого процессора $p = 1, \dots, P$ хранится список интервалов занятости $[t_{\text{start}}, t_{\text{end}})$.
- Глобально хранится информация о занятости памяти: для каждого момента времени (фактически в моменты стартов и завершений работ) известно, какие буферы активны.

Шаг 2. Для каждой работы $i \in \mathcal{R}$ и для каждого процессора p :

1. Найти наиболее раннее время t такое, что:

- процессор p свободен на интервале $[t, t + d_i)$ (нет пересечений с уже назначенными работами);
- для всех предшественников j работы i выполнено $s_j + d_j \leq t$;
- если работа i начнётся в момент t , то для всех моментов $\tau \in [t, t + d_i)$ суммарный объём активных буферов (включая буферы, создаваемые работой i) не превышает M .

2. Если такое t существует, запомнить для пары (i, p) возможное время старта $t_{i,p}$.

Для проверки третьего условия необходимо моделировать изменение состояния памяти: в момент старта работы i её буферы $j = 1, \dots, n_i$ занимают память r_{ij} , а освобождаются они только после того, как все потребители этих буферов завершат выполнение. В жадном алгоритме это условие проверяется приближённо, поскольку будущие потребители ещё не назначены. На практике достаточно потребовать, чтобы в момент старта t суммарный объём уже занятых буферов плюс объёмы всех буферов работы i не превышал M , а также чтобы в течение выполнения i не происходило превышения из-за освобождения буферов других работ (это контролируется через знание моментов освобождения). Подробный алгоритм проверки памяти выходит за рамки данного описания, однако в

реализации он основан на отслеживании моментов освобождения каждого буфера после завершения его последнего потребителя.

Шаг 3. Среди всех пар (i, p) , для которых найдено допустимое $t_{i,p}$, выбрать одну работу i^* и процессор p^* согласно выбранной эвристике (см. подраздел 3.2). Если ни одной допустимой пары нет, задача не имеет решения (либо M слишком мало, либо граф содержит тупиковые зависимости).

Шаг 4. Зафиксировать работу i^* на процессоре p^* со стартом t_{i^*,p^*} . Обновить расписание: добавить интервал занятости $[t, t + d_{i^*})$ для процессора p^* , зафиксировать момент старта $s_{i^*} = t$ и завершения $c_{i^*} = t + d_{i^*}$. Обновить состояние памяти: в момент s_{i^*} активировать буферы работы i^* , в момент c_{i^*} пока ничего не освобождать, т.к. буферы могут быть нужны потребителям.

Шаг 5. Удалить i^* из $\mathcal{U}$ и $\mathcal{R}$. Добавить в $\mathcal{R}$ все работы, у которых все предшественники уже находятся в $\mathcal{U}_{\text{назначенные}}$ (т.е. все предшественники поставлены в расписание). Если $\mathcal{U} \neq \emptyset$, перейти к шагу 2.

Шаг 6. По окончании работы алгоритма получаем полное расписание — для каждой работы определены процессор и время старта. Общая длительность (makespan) $L = \max_i (s_i + d_i)$.

3.2 Эвристики выбора работы

Ключевым элементом жадного алгоритма является правило выбора работы из множества готовых. В данной работе рассматриваются две эвристики: **EFT** (Earliest Finish Time) и **STS** (Smallest Time Space).

3.2.1 Эвристика EFT

Эвристика EFT ориентирована на минимизацию времени завершения работы. После того как для каждой готовой работы i и каждого процессора p определено наиболее раннее допустимое время старта $t_{i,p}$, вычисляется время завершения:

$$f_{i,p} = t_{i,p} + d_i.$$

Затем выбирается пара (i^*, p^*) , дающая наименьшее $f_{i,p}$. В случае равенства предпочтение отдается работе с наименьшим номером (или процессору с наименьшим номером, если несколько процессоров дают одинаковое время завершения для одной работы).

Эта эвристика широко известна в литературе по списочному планированию [? ?] и отличается простотой вычислений.

3.2.2 Эвристика STS

Эвристика STS (Smallest Time Space) введена для учёта влияния работы на использование памяти в будущем. Оценка работы складывается из двух компонент.

Первая компонента учитывает собственные буферы работы:

$$S_i^{(1)} = d_i \cdot \sum_{j=1}^{n_i} r_{ij}.$$

То есть произведение длительности работы на суммарный объём создаваемых ею буферов.

Вторая компонента учитывает «нагрузку» на память, создаваемую потомками работы. Для каждого буфера j работы i определим множество его потребителей $Cons(i, j) = \{k \mid (i, k, j) \in E\}$. Пусть $d_{total}(i, j)$ — суммарная длительность всех работ-потребителей этого буфера (каждый потребитель учитывается один раз, даже если он использует несколько буферов). Тогда

$$S_i^{(2)} = \sum_{j=1}^{n_i} \left(r_{ij} \cdot \sum_{k \in Cons(i, j)} d_k \right).$$

Общая оценка работы i равна

$$\phi(i) = S_i^{(1)} + S_i^{(2)}.$$

Алгоритм выбирает работу с наименьшим значением $\phi(i)$. Если несколько работ имеют одинаковую оценку, то среди них выбирается работа с наименьшим временем завершения (т.е. срабатывает правило EFT). Идея эвристики STS заключается в том, что работа, которая создаёт большие буферы и эти буферы нужны долго выполняющимся потомкам, потребляет память на длительное время, поэтому её выгодно выполнить позже (чтобы не блокировать память). Соответственно, на ранних шагах следует выбирать работы с малым $\phi(i)$.

4 Сведение к задаче целочисленного линейного программирования

Входные данные

Каждая работа i ($i = 1, \dots, n$) характеризуется длительностью $d_i > 0$ и набором буферов, которые она создаёт:

$$V = \left\{ (i, n_i, r_{i1}, r_{i2}, \dots, r_{in_i}) \right\}_{i=1}^n,$$

где n_i — количество буферов работы i , а $r_{ij} > 0$ — объём памяти, занимаемый j -м буфером. Дуги графа задают передачу данных:

$$E = \{(i, k, j)\}, \quad j \in [1, n_i],$$

т.е. дуга (i, k, j) означает, что j -й буфер работы i используется работой k (является её входным буфером). Все времена d_i целые, допустимый объём памяти равен M , число процессоров P .

Переменные

- s_i — время старта работы i (целочисленная переменная).
- m_{ij} — бинарная переменная, $m_{ij} = 1$ если $s_i \leq s_j$, иначе 0 ($i, j = 1, \dots, n$).
- p_{ri} — бинарная переменная, $p_{ri} = 1$ если работа i назначена на процессор r ($r = 1, \dots, P$).
- t_{ij} — бинарная переменная, $t_{ij} = 1$ если работы i и j выполняются на одном процессоре.
- l — целочисленная переменная, общая длительность расписания (makespan).
- Для каждого буфера (i, j) ($j = 1, \dots, n_i$) вводятся целочисленные переменные:

$$\text{start}_{ij}, \quad \text{max_end}_{ij}.$$

start_{ij} совпадает со стартом работы i , а max_end_{ij} — максимальное время завершения среди всех потребителей этого буфера.

- Для каждой пары буферов (i, j) и (p, q) вводятся бинарные переменные:

$$z_{ij}^{pq}, z_{ij}^{pq}, a_{ij}^{pq}, aa_{ij}^{pq}.$$

Они служат для моделирования пересечений интервалов активности буферов.

Вспомогательная константа

$$D = \sum_{i=1}^n d_i,$$

которая заведомо больше любого возможного makespan.

Ограничения, связывающие m_{ij} и s_i

Для всех i, j :

$$m_{ii} = 1, \tag{1}$$

$$D m_{ij} - s_j + s_i \geq 0, \tag{2}$$

$$s_j - s_i - D m_{ij} \geq 1 - D. \tag{3}$$

Эти условия эквивалентны: $m_{ij} = 1 \Leftrightarrow s_i \leq s_j$.

Назначение на процессоры

Для всех i и r :

$$\sum_{r=1}^P p_{ri} = 1, \tag{4}$$

$$p_{ri} \in \{0, 1\}.$$

Переменные t_{ij} (один процессор)

Для всех i, j, k ($i \neq j$) и всех r :

$$t_{ij} \geq p_{ri} + p_{rj} - 1, \tag{5}$$

$$t_{ij} \leq p_{ri} - p_{rj} + 1, \tag{6}$$

$$t_{ij} \in \{0, 1\}.$$

При этом $t_{ij} = 1$ тогда и только тогда, когда $p_{ri} = p_{rj} = 1$ для некоторого r .

Зависимости по графу

Для каждой дуги $(i, k, j) \in E$ (т.е. j -й буфер работы i потребляется работой k):

$$s_k \geq s_i + d_i. \quad (7)$$

Определение start_{ij} и max_end_{ij}

Для каждого буфера (i, j) :

$$\text{start}_{ij} = s_i, \quad (8)$$

$$\text{max_end}_{ij} \geq s_k + d_k \quad \text{для всех } k \text{ таких, что } (i, k, j) \in E. \quad (9)$$

Таким образом, max_end_{ij} — максимум моментов завершения всех потребителей буфера.

Взаимное исключение на одном процессоре

Для всех $i \neq j$:

$$s_i - s_j + D m_{ij} - D t_{ij} \geq -D + d_j, \quad (10)$$

$$s_j - s_i - D m_{ij} - D t_{ij} \geq -2D + d_i. \quad (11)$$

Если $t_{ij} = 1$ (работы на одном процессоре), то эти ограничения вынуждают $s_j \geq s_i + d_i$ при $m_{ij} = 1$ и $s_i \geq s_j + d_j$ при $m_{ij} = 0$. При $t_{ij} = 0$ ограничения выполняются автоматически.

Учёт пересечений буферов (для ограничения памяти)

Для каждой пары буферов (i, j) и (p, q) введём вспомогательные бинарные переменные z_{ij}^{pq} , zz_{ij}^{pq} , a_{ij}^{pq} , aa_{ij}^{pq} , связанные следующими линейными ограничениями:

$$\text{start}_{pq} - \text{start}_{ij} - (D + 1) z_{ij}^{pq} \geq -D, \quad (12)$$

$$(D + 1) a_{ij}^{pq} + (D + 1) z_{ij}^{pq} + \text{start}_{ij} - \text{max_end}_{pq} \geq 0, \quad (13)$$

$$\text{start}_{pq} - \text{max_end}_{ij} - (D + 1) zz_{ij}^{pq} \geq -D, \quad (14)$$

$$(D + 1) aa_{ij}^{pq} + (D + 1) zz_{ij}^{pq} + \text{max_end}_{ij} - \text{max_end}_{pq} \geq 0. \quad (15)$$

Переменные a_{ij}^{pq} и aa_{ij}^{pq} интерпретируются следующим образом:

- $a_{ij}^{pq} = 1$, если буфер (p, q) активен в момент старта буфера (i, j) (т.е. start_{ij} лежит внутри интервала активности (p, q));
- $aa_{ij}^{pq} = 1$, если буфер (p, q) активен в момент завершения буфера (i, j) (т.е. max_end_{ij} лежит внутри интервала активности (p, q)).

Интервал активности буфера (p, q) — это $[\text{start}_{pq}, \text{max_end}_{pq})$.

Ограничение на суммарную память

Для каждого буфера (i, j) суммарный объём памяти, занятый в момент start_{ij} (соответственно max_end_{ij}), не должен превышать M :

$$\sum_{(p,q)} r_{pq} a_{ij}^{pq} \leq M, \quad (16)$$

$$\sum_{(p,q)} r_{pq} aa_{ij}^{pq} \leq M, \quad (17)$$

где сумма берётся по всем буферам, присутствующим в графе. Благодаря определению a_{ij}^{pq} и aa_{ij}^{pq} , эти ограничения гарантируют, что пиковое потребление памяти в любой момент (все точки вида start_{ij} или max_end_{ij}) не превышает M .

Целевая функция

Минимизируем общую длительность расписания:

$$\min l, \quad (18)$$

$$l \geq s_i + d_i \quad \forall i. \quad (19)$$

Итоговая задача

Минимизировать l

при ограничениях (1)–(3), (4), (5)–(6), (7), (8), (9),

(10)–(11), (12)–(15), (16)–(17), (19),

переменные $s_i, \text{start}_{ij}, \text{max_end}_{ij}, l \in \mathbb{Z}_{\geq 0}$, $m_{ij}, p_{ri}, t_{ij}, z_{ij}^{pq}, z_{ij}^{pq}, a_{ij}^{pq}, aa_{ij}^{pq} \in \{0, 1\}$

Число переменных и ограничений

Пусть n — число работ, n_i — число буферов работы i , $B = \sum_i n_i$ — общее число буферов, P — число процессоров. Тогда:

- m_{ij} : n^2 бинарных.
- s_i : n целых.
- p_{ri} : Pn бинарных.
- t_{ij} : n^2 бинарных.
- $\text{start}_{ij}, \text{max_end}_{ij}$: $2B$ целых.
- $z_{ij}^{pq}, zz_{ij}^{pq}, a_{ij}^{pq}, aa_{ij}^{pq}$: $4B^2$ бинарных.
- l : одна целая.

Количество ограничений составляет $O(n^2 + Pn + |E| + B^2)$.

5 Экспериментальное исследование

5.1 Предмет и цели исследования

Предметом исследования являются предложенные подходы к решению задачи: жадный алгоритм с эвристиками EFT и STS; подход на основе ЦЛП. Целями исследования является сравнительный анализ подходов по следующим критериям:

- качество получаемого решения (т.е. значение целевой функции на расписании);
- масштабируемость по времени поиска решения при увеличении размерности входных данных.

5.2 Классы графов работ

Используются три класса ориентированных ациклических графов:

5.2.1 Слоистые графы

Слоистые графы (рис. 1) характерны для многоэтапных вычислений (цифровая обработка сигналов, научные расчёты). Вершины разбиты на последовательные слои, рёбра в основном соединяют вершины соседних слоёв, но допускаются «перескоки» через несколько слоёв.

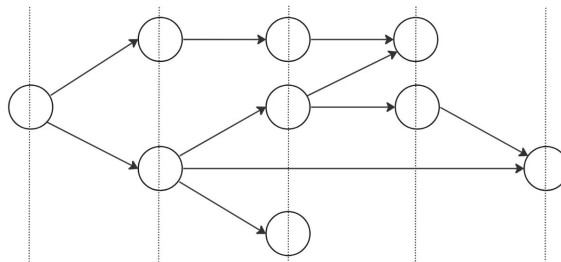


Рис. 1: Пример слоистого графа

5.2.2 Случайные графы

Случайные графы (рис. 2) генерируются с ограничениями: у каждой вершины не более двух предков и не более шести потомков. Такие графы встречаются, например, при реализации LU- и QR-разложений.

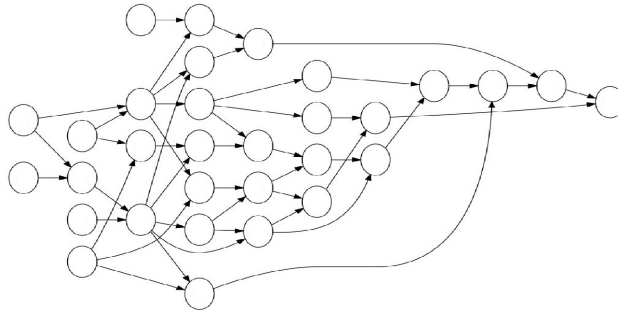


Рис. 2: Пример случайного графа

5.2.3 Треугольные графы

Треугольные графы (рис. 10) — частный случай слоистых графов, моделирующий метод Гаусса–Жордана. Они обладают регулярной структурой: каждый слой содержит на одну вершину меньше предыдущего, рёбра соединяют только соседние слои. Используются только для больших размеров задач.

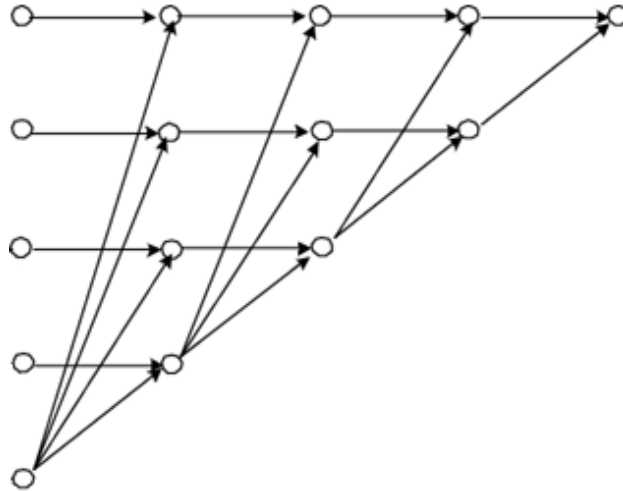


Рис. 3: Пример треугольного графа

5.3 Наборы входных данных

Для каждого из двух исследований (качество и масштабируемость) используются отдельные наборы графов, различающиеся размерностью. Все графы имеют следующие общие параметры:

- Длительность работы d_i — целое число, равномерно распределённое в диапазоне $[10, 100]$.
- Вес каждого буфера — целое число, равномерно распределённое в диапазоне $[1, 100]$.
- Для каждой работы используются два варианта формирования буферов:
 1. **Один буфер:** все исходящие рёбра объединены в один буфер указанного веса.
 2. **Корневое равномерное распределение:** количество буферов равно $\lceil \sqrt{\text{outdeg}} \rceil$, рёбра распределяются по буферам максимально равномерно.

5.3.1 Малые графы

- **Слоистые графы:** все размеры $n = 10, 11, \dots, 21$ (по одному графу на размер).
- **Случайные графы:** все размеры $n = 10, 11, \dots, 21$ (по одному графу на размер).

5.3.2 Большие графы

- **Слоистые графы:** по одному графу для каждого размера $n \in \{55, 210, 496, 990\}$.
- **Случайные графы:** по одному графу для каждого размера $n \in \{55, 210, 496, 990\}$.
- **Треугольные графы:**
 - Для оценки качества: по одному графу для каждого размера $n \in \{55, 210, 496, 990\}$
 - Для оценки масштабируемости: расширенная сетка размеров - по одному графу для каждого размера $n \in \{55, 105, 153, 210, 300, 496, 741, 990\}$

5.3.3 Сетка параметров (P, M)

Для каждого графа предварительно определяются:

- $P_{\max}$ — максимальное число процессоров, которое может эффективно использовать жадный алгоритм на данном графе при заведомо избыточной памяти.
- $M_{\max}$ — максимальная пиковая память, возникающая при выполнении расписания с заведомо избыточным числом процессоров.

Число процессоров P :

- Если $P_{\max} \geq 50$, то $P \in \{4, \lfloor (4 + P_{\max})/2 \rfloor, P_{\max}\}$.
- Если $P_{\max} < 50$, то $P \in \{2, \lfloor (2 + P_{\max})/2 \rfloor, P_{\max}\}$.

Лимит памяти M :

- Для малых графов ($n \leq 21$): $M_{\min} = 1$, поиск ведётся с шагом 1 до первого допустимого расписания.
- Для больших графов ($n \geq 55$): $M_{\min} = \lfloor M_{\max}/10 \rfloor$, шаг поиска $\Delta M = \lfloor M_{\max}/100 \rfloor$.
- В обоих случаях используются три уровня: $M \in \{M_{\min}, \lfloor (M_{\min} + M_{\max})/2 \rfloor, M_{\max}\}$.

Таким образом, для каждого исходного графа формируется 9 вариантов входных данных (3×3 комбинации P и M), на которых проводятся запуски.

5.4 Результаты экспериментов

Эксперименты проводились на компьютере со следующими характеристиками:

- ЦП: Apple M1
- Количество ядер: 8;
- Объём ОЗУ: 16 Гбайт;
- ОС: macOS Sonoma 14.5.

Выбор решателя и схема параллельного запуска В качестве решателя задач целочисленного линейного программирования используется SCIP. Выбор обоснован в работе А.К. Сенюшкиной [?], где проведено сравнительное исследование производительности свободно доступных решателей (GLPK, CBC, SCIP) на графах рассматриваемых классов; SCIP показал наилучшие результаты по времени поиска оптимального решения.

Из-за значительной зависимости времени работы решателя от упорядочения переменных и ограничений, а также от добавления транзитивных рёбер (см. [?]), применяется метод «параллельного запуска с остановкой по первому успеху». Для каждого входного графа формируются 8 вариантов .lp-файлов, соответствующих комбинациям:

- 4 упорядочения вершин и рёбер: `default`, `up_right`, `down_left`, `tiers`;
- 2 варианта графа: с добавленными транзитивными ограничениями (флаг `-tr`) и без них.

(Упорядочение `reverse_tiers` не используется из-за ограниченности числа доступных процессорных ядер.)

Затем скрипт `run_scip2.sh` одновременно запускает 8 экземпляров SCIP по одному на каждый вариант. Как только хотя бы один экземпляр завершается с нахождением оптимального решения (статус `optimal solution found`), все остальные процессы принудительно завершаются. Это позволяет избежать неоправданного ожидания на «медленных» упорядочениях и эффективно использовать доступные вычислительные ресурсы. Лимит времени на каждый экземпляр задаётся в файле `only_time.set` (в экспериментах использовалось 3600 секунд).

5.4.1 Малые графы

Исследование масштабируемости

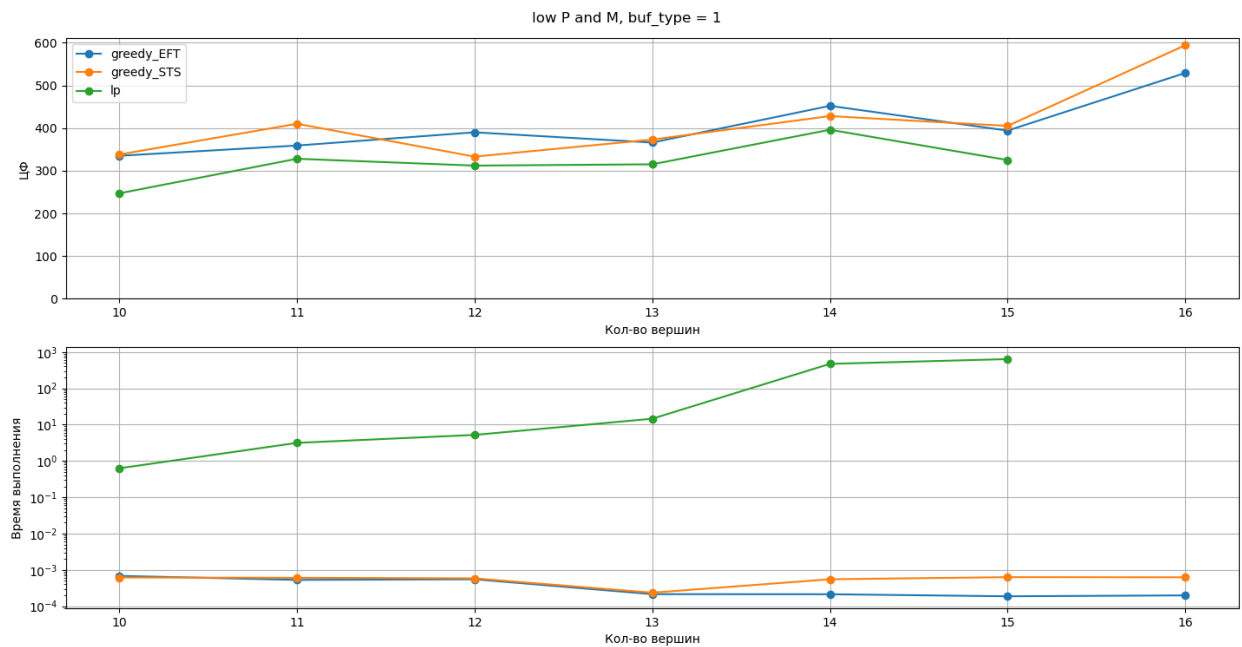


Рис. 4: описание 1

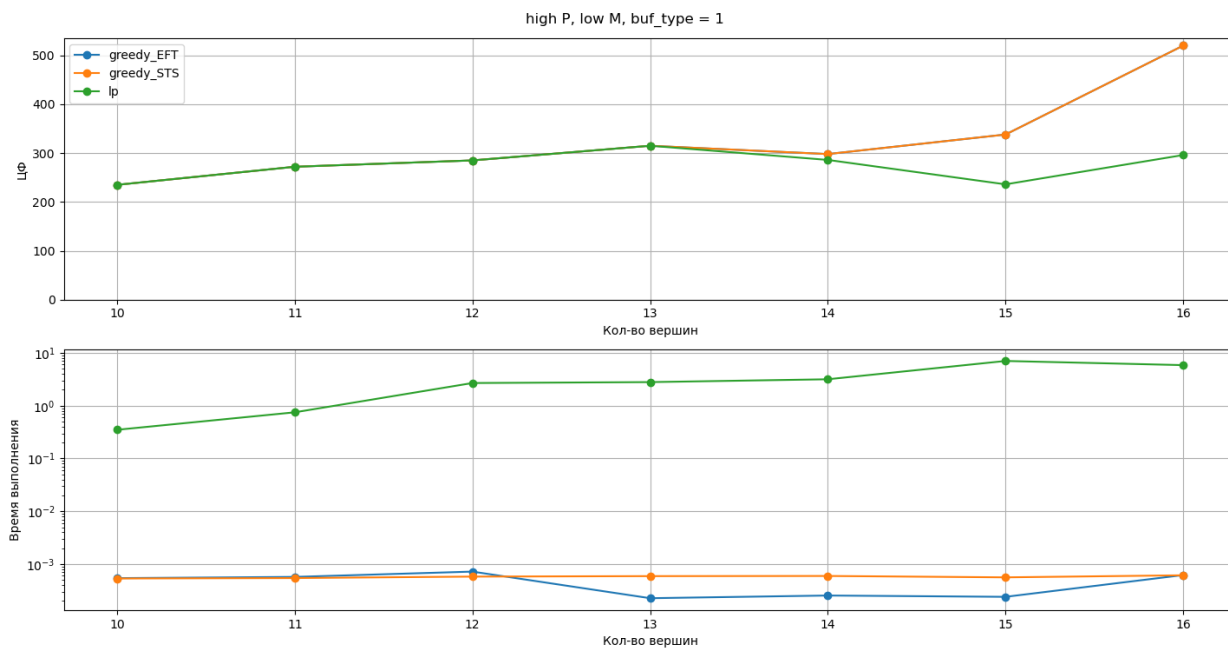


Рис. 5: описание 2

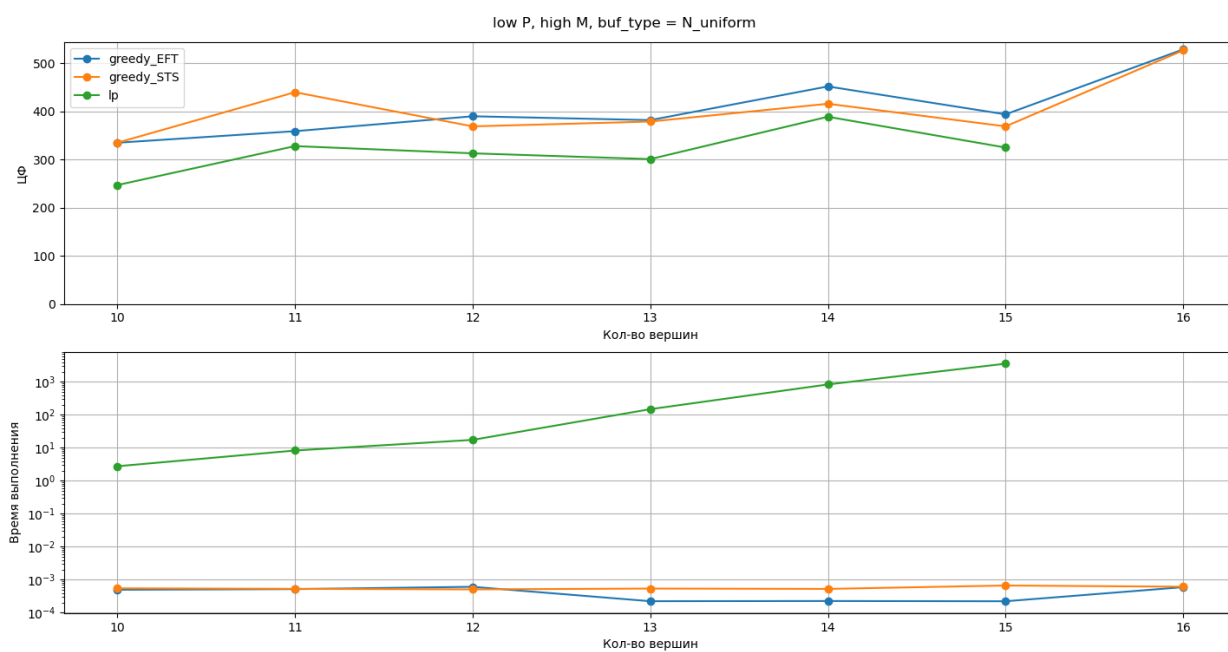


Рис. 6: описание 3

Исследование качества решения

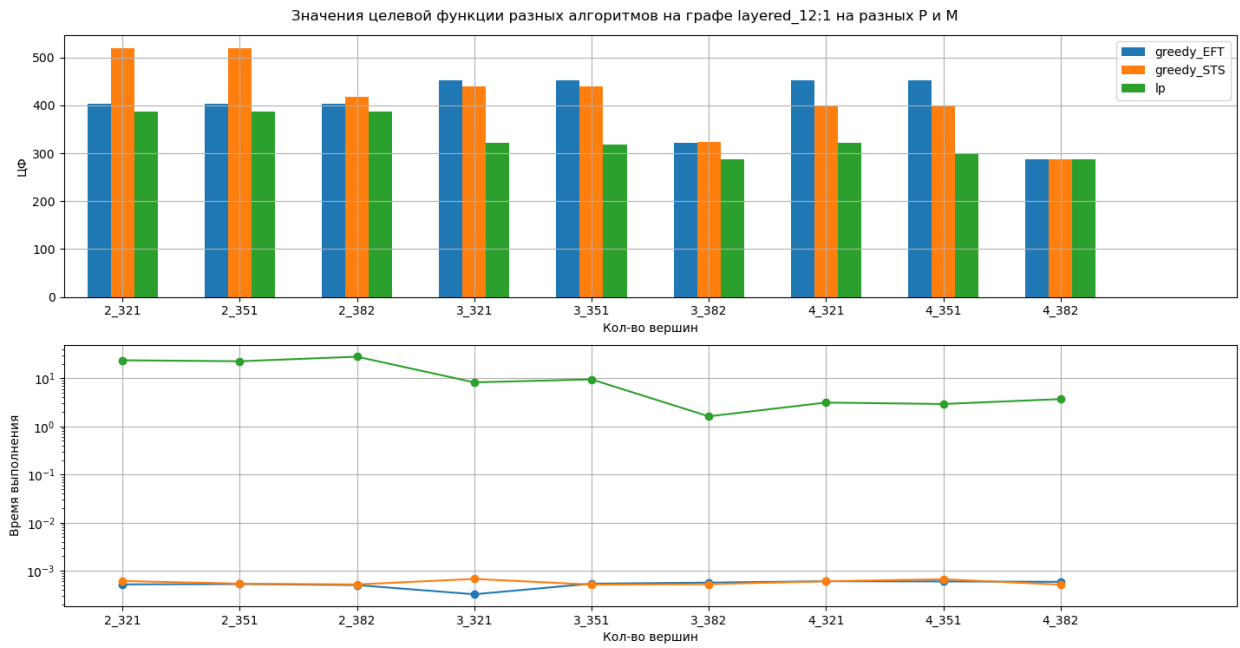


Рис. 7: описание 4

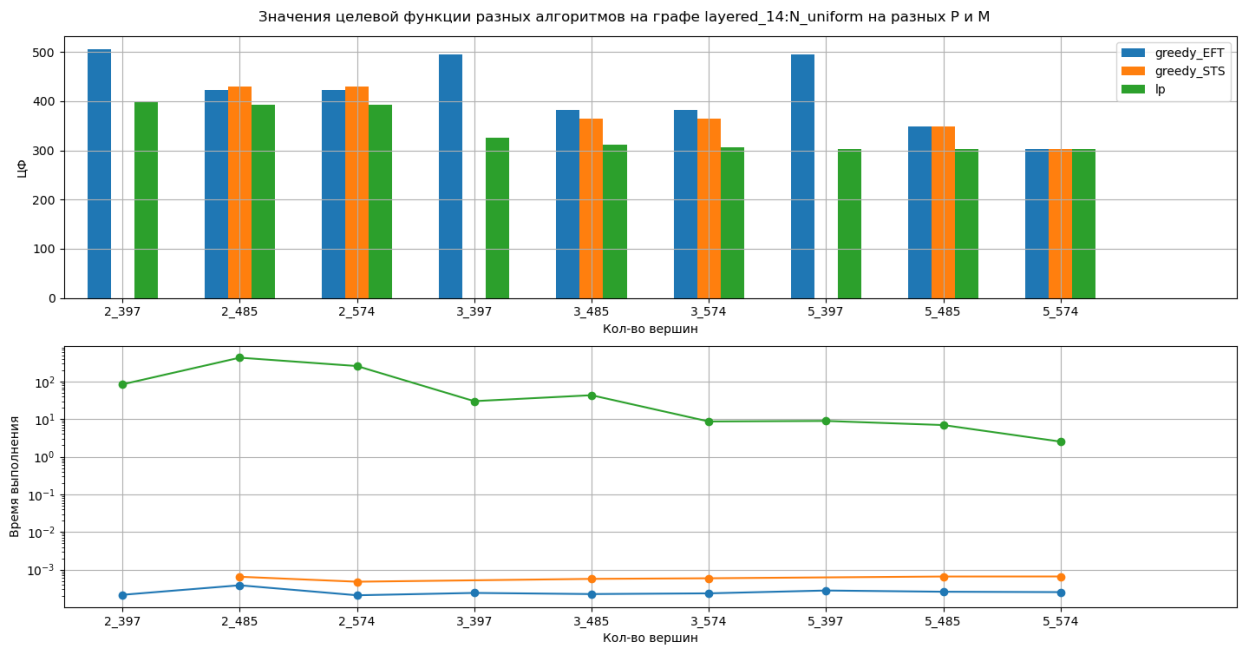


Рис. 8: описание 5

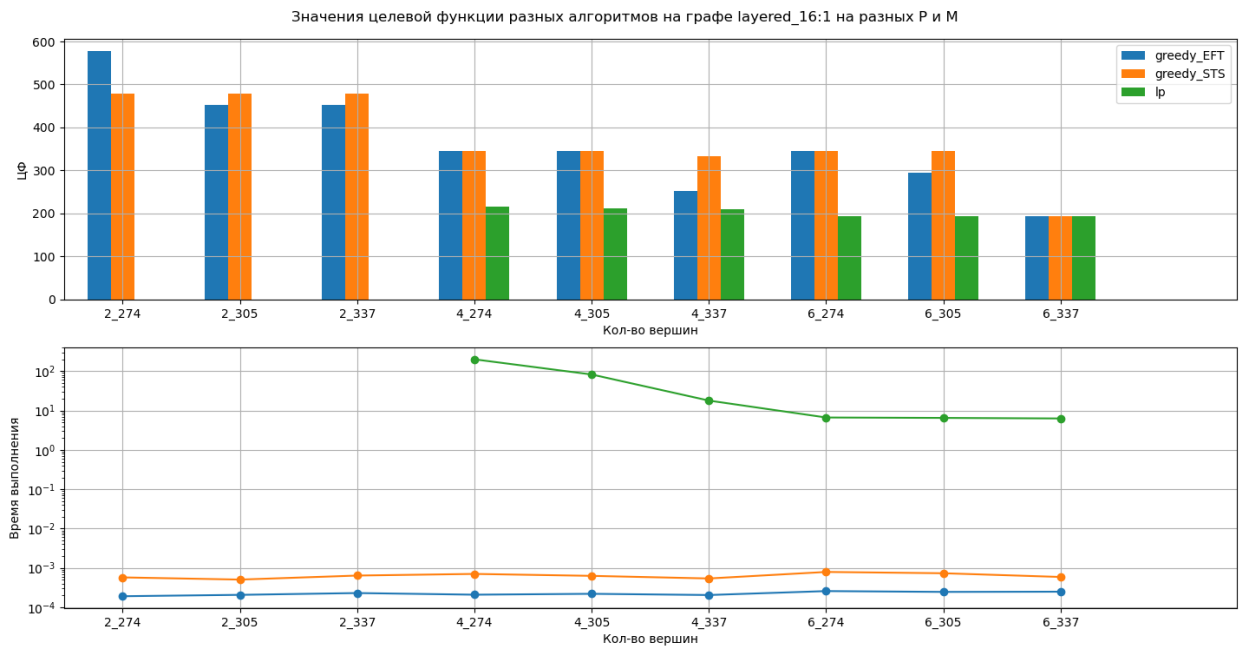


Рис. 9: описание 6

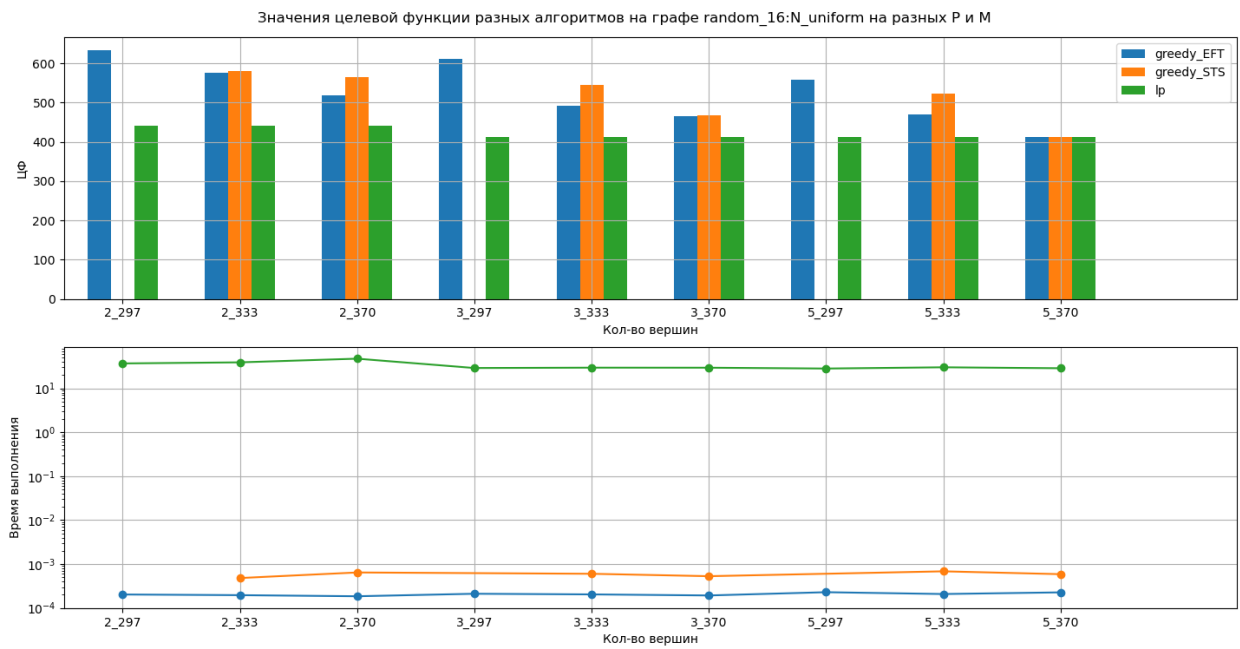


Рис. 10: описание 7

6 Описание программной реализации

6.1 Структура входного и выходного файлов

6.1.1 Входной файл (описание графа работ)

Входной файл имеет расширение `.txt` и содержит строки в кодировке UTF-8. Первая строка является заголовком и содержит названия полей:

```
prog_id prog_time weight_1: child_1 ... child_n, weight_2: child_1 ... child_n, ...
```

Каждая последующая строка описывает одну работу (вершину графа) и имеет следующий формат:

```
<id> <time> <буфер_1>, <буфер_2>, ..., <буфер_k>
```

где

- `<id>` — целочисленный идентификатор работы (от 0 до $n - 1$);
- `<time>` — целочисленная длительность выполнения работы;
- `<буфер_i>` имеет вид `<вес>: <список потомков>`, где `<вес>` — целое число, задающее объём памяти, занимаемый данным буфером, а `<список потомков>` — разделённые пробелами идентификаторы работ-потребителей этого буфера. Если у работы нет исходящих рёбер, в строке указывается единственный буфер 0: (нулевой вес, пустой список потомков).

Буферы в строке разделяются запятой с пробелом (‘, ‘). Внутри одного буфера после двоеточия может быть перечислено несколько потомков, что означает, что все они используют один и тот же буфер (и, следовательно, разделяют его данные). Пример строки для работы с двумя буферами:

```
2 36 10: 3 6 7, 73: 4 8
```

Это означает, что работа 2 длится 36 единиц времени, создаёт два буфера: первый весом 10 используется работами 3, 6, 7; второй весом 73 используется работами 4 и 8.

6.1.2 Выходной файл (расписание)

Результат работы жадного алгоритма или решателя ЦЛП сохраняется в файл формата JSON с расширением `.json`. Файл содержит один объект, ключом которого является имя входного графа (без расширения), а значением — объект с полями, описывающими расписание. Структура JSON-объекта:

```
{
  "имя_графа": {
    "makespan": целое,
    "runtime_sec": число с плавающей точкой,
    "size": целое,
    "schedule": {
      "id_вершины": {
        "processor": целое,
        "start": целое,
        "finish": целое
      },
      ...
    }
  }
}
```

Поля означают:

- **makespan** — общая длительность расписания (максимальное время завершения среди всех работ);
- **runtime_sec** — реальное время работы алгоритма в секундах;
- **size** — число вершин в графе;
- **schedule** — словарь, где каждому идентификатору вершины соответствует словарь с тремя полями:
 - **processor** — номер процессора (начиная с 0), на котором выполняется работа;

- `start` — время начала выполнения;
- `finish` — время завершения (`start` + длительность работы).

6.2 Реализация жадного алгоритма

Программная реализация жадных алгоритмов выполнена на языке C++ (стандарт C++17). В основе реализации лежит модульная архитектура, позволяющая легко добавлять новые эвристики и конфигурировать параметры (число процессоров, лимит памяти). Основные классы и их взаимосвязи показаны на диаграмме (рис. 11).

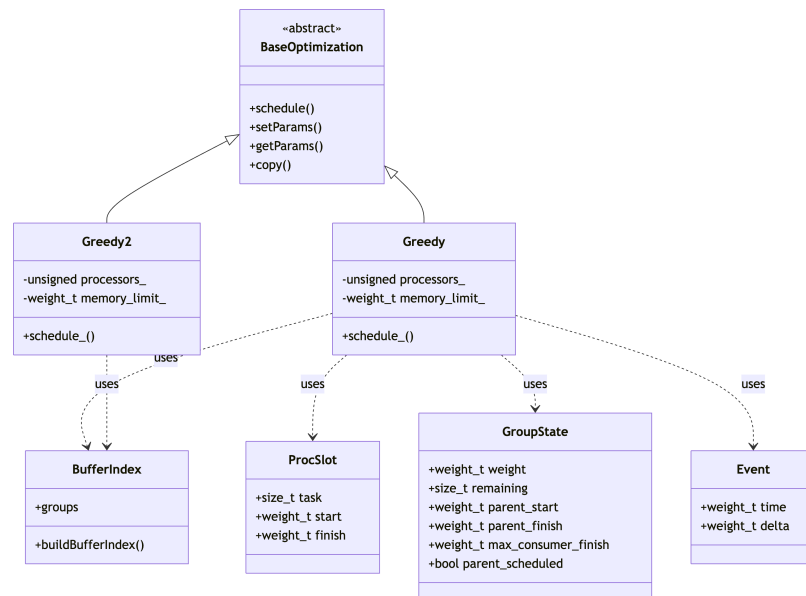


Рис. 11: Диаграмма классов жадного алгоритма

Базовый класс BaseOptimization Абстрактный класс, задающий интерфейс для всех алгоритмов оптимизации расписания. Содержит виртуальные методы `schedule()` (построение расписания), `setParams()/getParams()` (управление параметрами) и `copy()` (клонирование). Наследники должны реализовать конкретный метод `schedule_()`.

Класс Greedy (EFT) Реализует жадный алгоритм, использующий эвристику **Earliest Finish Time (EFT)**. На каждом шаге для каждой готовой работы на каждом про-

цессоре определяется наиболее раннее время старта с учётом уже назначенных работ на процессоре и зависимостей по данным. Затем вычисляется время завершения $f = start + duration$. Выбирается работа и процессор, дающие минимальное f ; при равенстве предпочтение отдаётся работе с меньшим идентификатором (и процессору с меньшим номером). Ключевая функция — `earliestStartOnProcessor2()` — просматривает интервалы занятости процессора и находит подходящее окно.

Класс Greedy2 (STS) Реализует эвристику **Smallest Time Space (STS)**, учитывающую как длительность работы, так и её влияние на будущую занятость памяти. Для каждой работы i предварительно вычисляется оценка:

$$\phi(i) = d_i \cdot \sum_j w_{ij} + \sum_j \left(w_{ij} \cdot \sum_{k \in consumers(i,j)} d_k \right),$$

где w_{ij} — объём j -го буфера, $consumers(i, j)$ — множество работ-потребителей этого буфера. В процессе планирования работа с наименьшим $\phi(i)$ получает приоритет; при равенстве оценок выбирается работа с минимальным f (EFT). Вычисление оценки вынесено в отдельные переменные `output_buffers_weight_sum` и `weighted_duration_sum` внутри основного цикла.

Вспомогательные структуры

- **ProcSlot** — описывает отрезок времени, занятый работой на процессоре (интервал $[start, finish)$).
- **GroupState** — состояние буфера: объём памяти, число оставшихся потребителей, время старта/завершения родительской работы и максимальное время завершения среди уже назначенных потребителей.
- **Event** — событие изменения занятости памяти (выделение или освобождение), используется при проверке допустимости по памяти.
- **BufferIndex** — индекс буферов, построенный по входному графу: для каждой работы хранит список её буферов, а для каждого буфера — список потребителей и объём. Используется для инициализации состояний **GroupState**.

Учёт ограничения памяти Для каждой кандидатной работы проверяется, не превысит ли пиковое использование памяти заданный лимит M при помещении работы в предполагаемое время старта. Для этого:

1. Формируется список событий (выделение/освобождение) на основе уже зафиксированных буферов (поля `parent_scheduled`).
2. Добавляются события выделения для буферов самой кандидатной работы в момент её старта.
3. Для каждого родительского буфера, у которого кандидат является последним потребителем, добавляется событие освобождения в момент $\max(\text{finish}_{\text{parent}}, \text{finish}_{\text{consumer}}, \text{finish}_{\text{candidate}})$.
4. События сортируются по времени, вычисляется скользящая сумма занятой памяти; если в любой момент сумма превышает M , размещение отвергается.

Для графов с большим числом работ эта проверка является наиболее вычислительно затратной, поэтому она оптимизирована за счёт предварительного упорядочивания событий и раннего выхода при превышении лимита.

Особенности реализации

- Все времена и объёмы памяти представлены целочисленным типом `weight_t` (псевдоним `long long`).
- Процессоры нумеруются с 0 до $P - 1$, расписание на каждом процессоре хранится как отсортированный вектор интервалов `ProcSlot`.
- Множество готовых работ поддерживается в `std::set` (автоматическая сортировка по идентификатору).
- Для ускорения поиска стартового окна на процессоре используется линейный проход по интервалам — в силу того, что число интервалов на процессор не превышает числа работ, а общее число работ редко превышает несколько тысяч, этого достаточно.

Объём написанного кода для жадных алгоритмов составляет около 500 строк (файлы `Greedy.cpp`, `Greedy2.cpp` и заголовочные файлы). Программная реализация доступна в репозитории проекта (ссылка приведена в соответствующем разделе).

6.3 Реализация подхода на базе ЦЛП

Подход, описанный в разделе ??, реализован в виде набора скриптов на языке Python (версия 3.9) и Bash. Все программы, включая исходные тексты и входные данные, доступны в репозитории [?]. Основные компоненты:

- `make_input_times_step2_2.py` – транслятор, преобразующий входной граф работ (в новом формате, см. подраздел ??) в описание задачи ЦЛП на входном языке решателя SCIP (файл с расширением `.lp`). Для каждого графа генерируется 10 вариантов `.lp`-файлов: 5 упорядочений переменных и ограничений (`default`, `tiers`, `reverse_tiers`, `up_right`, `down_left`) с добавлением транзитивных рёбер графа (флаг `-tr`) и без них. Структура `.lp`-файла соответствует распределению, приведённому в табл. 1.
- `run_many_make_inputs_time_for_LP.py` – автоматизирует вызов транслятора для всех входных графов из каталога экспериментов. Из имени каждого файла извлекаются значения числа процессоров P и лимита памяти M (см. подраздел 5.3), и эти параметры подставляются в глобальные переменные скрипта `make_input_times_step2_2.py`.
- `run_many_solvers.py` – управляет выполнением серии экспериментов через обёрточный скрипт `run_scip2.sh`. Ограничение времени работы решателя задаётся в файле `only_time.set` (параметр `limits/time = 3600`).
- `run_scip2.sh` – реализует параллельный запуск 8 экземпляров SCIP. Логи работы сохраняются в отдельные каталоги.
- `file_reader_and_making_json.py` – извлекает из логов SCIP значения переменных:
 - s_i (время старта работы i),

- p_{ri} (номер процессора, назначенного работе i),
- вычисляет makespan как $\max_i(s_i + d_i)$.

Результат записывается в JSON-файл, полностью совместимый по структуре с выходными файлами жадных алгоритмов (см. подраздел ??).

Для генерации входных графов и подготовки наборов экспериментов использованы дополнительные скрипты:

- `layered_generator.py`, `random_generator.py`, `triadag_generator.py` – создают графы в «старом» формате (вершина, вес, список потомков) без учёта буферов.
- `converter.py` – преобразует «старые» графы в новый формат с буферами: назначает случайные длительности работ ($\in [10, 100]$) и веса буферов ($\in [1, 100]$), формирует два варианта группировки рёбер в буферы – один буфер (`_1.txt`) и равномерное корневое распределение (`_N_uniform.txt`).
- `finding_M_and_P.py` – для каждого полученного графа вычисляет на практике максимальное используемое число процессоров $P_{\max}$ и максимальную пиковую память $M_{\max}$ (при заведомо избыточных ресурсах). Затем создаёт 9 вариантов входных файлов (3 значения $P \times 3$ значения M) согласно сетке, описанной в подразделе 5.3.

Скрипты `run_scip2.sh` и `only_time.set` обеспечивают воспроизводимость экспериментов и возможность ограничения времени счёта. Все сгенерированные расписания (в формате JSON) и логи сохраняются в структурированном виде, что позволяет автоматизировать последующий анализ результатов.

6.4 Визуализатор временной диаграммы расписания

Для наглядного представления построенных расписаний (как полученных жадными алгоритмами, так и с помощью ЦЛП) разработан скрипт `gantt_and_memory.py` на языке Python с использованием библиотеки `matplotlib`. Он создаёт совмещённое изображение, состоящее из двух диаграмм:

Таблица 1: Распределение переменных и ограничений по секциям .lp-файла

Секция	Переменные / ограничения	Пояснение
Integer	$l, s_i, \text{start}_{i,j}, \text{max_end}_{i,j}$	Целевая переменная, времена старта работ, начала и макс. завершения буферов
Binary	$m_{ij}, p_{ri}, t_{ij}, z_{ij}^{pq}, z_{ij}^{pq}, a_{ij}^{pq}, aa_{ij}^{pq}$	Отношения порядка, назначение на процессоры, пересечения буферов
Subject to	(2)–(3), (7), (10)–(11), (12)–(15) и др.	Основные линейные ограничения
Bounds	$m_{ii} = 1, m_{ij} = 1$ для $(i, j) \in E$ (включая транзитивные)	Фиксированные значения переменных предшествования
Minimize	l	Целевая функция

1. **Диаграмма Ганта** – отображает загрузку процессоров во времени. Каждая работа изображается цветным прямоугольником, внутри которого указан её идентификатор. По горизонтальной оси отложено время, по вертикальной – номера процессоров.
2. **График использования памяти** – показывает, как меняется суммарный объём занятой памяти в процессе выполнения расписания. По вертикальной оси отложен объём памяти, по горизонтальной – время. Дополнительно красной горизонтальной линией отмечается заданный лимит памяти M .

Входные данные:

- JSON-файл с расписанием (формат описан в подразделе ??), содержащий для каждой работы номер процессора, время старта и завершения.
- Исходный TXT-файл графа работ в новом формате (с буферами), необходимый для определения, какие буферы активны в каждый момент времени.

Ключевые компоненты реализации:

- Функции рисования стрелок (`draw_horiz_arrow`, `draw_vertical_arrow`, `draw_arc_arrow`, `draw_loop_arrow` и др.) – используют `FancyArrowPatch` из `matplotlib.patches`. Они проводят дуги и линии между работами, визуализируя зависимости по данным (рёбра графа). Тип стрелки зависит от взаимного расположения работ: горизонтальная, вертикальная, диагональная или дугообразная.
- Класс `ResourceVisualizer` – накапливает события изменения памяти (выделение и освобождение буферов) для каждого момента времени. Метод `visualize()` строит ступенчатую диаграмму, где каждый прямоугольник соответствует периоду активности конкретного буфера. Метод `add_horizontal_line()` добавляет линию заданного лимита памяти.
- Функция `get_task_coordinates` – вычисляет координаты работы на диаграмме Ганта по её времени старта, длительности и номеру процессора.

Выходные данные: Скрипт отображает полученное изображение на экране (при интерактивном запуске) и может сохранить его в файл (например, в формате PNG) с высоким разрешением (300 dpi). Пример такого изображения приведён на рис. ?? (вкл. в приложение или в раздел результатов).

Визуализатор активно используется при отладке алгоритмов, а также для качественного анализа полученных расписаний в ходе экспериментального исследования.

Заключение

Здесь будет заключение